

μ KV: Energy- and Thermal-Aware KV-Cache Eviction for LLM Inference on a Phone

Md Romyull Islam
Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

On a phone the KV cache, not compute, limits sustained LLM decode. The policies that bound it were built for server GPUs. Three properties of a real on-device engine break them. One cell array serves every head and layer, so a cell is freed only when every head and every layer drops it. SnapKV reports an 8.2 times smaller cache; on this engine it retains 68% of the cells at a matched budget and 94% at its own. Reading attention scores forces the FlashAttention-off path, so score-reading evictors decode at 0.12 to 0.30 times the full cache on the phone GPU. Compaction through the state API needs a second full context, and the OS kills Phi-3-mini at 16K. μ KV scores tokens inside the FlashAttention prefill graph through a small side node, selects one keep set for all heads and layers, and compacts in place. A reduce-only clock watchdog with CPU and GPU ladders keeps the vendor throttle unreachable. A per-request scheduler picks the GPU clock plan, the cache budget and the answer cap from the battery state, and two loops correct it. On a OnePlus 15, μ KV decodes 4.8 times faster than the full cache on the CPU and 1.23 times on the Adreno GPU, at 7% of the cells and 62% less energy. It ties the full cache on 392 retrieval cells and stays within 0.1 to 3.7 F1 on LongBench at 13% of the cache. Through a real discharge from 91 to 11%, its two lower tiers cut energy per request by 15% and 43%.

KEYWORDS

Mobile LLM inference, KV cache eviction, FlashAttention, thermal management, energy-aware scheduling, on-device AI

1 INTRODUCTION

Small instruction-tuned models now run interactively on flagship phones. For a sustained generation the wall is the KV cache, not the matrix multiply. Every decode step reads the whole cache once per layer. Past a thousand tokens the step is bound by DRAM bandwidth. The DRAM heats, the vendor thermal engine cuts the clock, and the battery drains. Cache size sets traffic, traffic sets power, power sets temperature, temperature sets the throttle, and the throttle sets throughput. A mobile cache policy must therefore be judged on quality, throughput, heat and energy together, on real hardware.

The policies that bound the cache were built for server GPUs and judged by perplexity alone: H2O [1], TOVA [3], SnapKV [4], StreamingLLM [2] and Ada-KV [6]. We ran each in its published configuration on a OnePlus 15 inside llama.cpp. Three properties of the engine broke them.

- **Budgets do not materialize.** The engine keeps one cell array shared by every head and layer. A cell is freed only when every

head and every layer drops it. SnapKV selects per head and reports an 8.2 times smaller cache at 16K inputs [4]. At a matched budget of 1024 per head on this engine it keeps 68% of the cells on Llama-3.2-1B and 85% on Phi-3-mini. At its own budget of 2048 per head it keeps 94%. H2O reports a 5 to 10 times smaller cache at a 20% budget [1]; at that budget here it keeps 92 to 99.8%.

- **Scores cost the fast path.** A policy that reads post-softmax attention cannot use the fused FlashAttention kernel. Every score-reading evictor decodes at 0.12 to 0.30 times the full cache on the phone GPU. Evicting makes it slower.
- **Compaction doubles memory.** The state API restores survivors into a second context. Phi-3-mini at 16K needs two 6.4 GB caches plus 2.4 GB of weights, and the OS kills it. The same call fails on a 24 GB desktop GPU.

μ KV is an evictor designed around these facts, with the two controllers a phone needs beside it (Figure 1). It computes selection scores inside the FlashAttention prefill graph through a small side node. It selects one keep set of K cells for all heads and layers, so eviction frees cells. It compacts in place: survivors slide into a dense prefix inside the tensors prefill already allocated. A reduce-only watchdog steps the CPU and GPU clocks down along temperature ladders anchored at the measured vendor triggers. A per-request scheduler picks the GPU clock plan, the cache budget and the answer cap from the battery state. It predicts the request's cost from a measured table, and two loops correct its lever when time or energy misses.

We evaluate on a OnePlus 15 (Snapdragon 8 Elite Gen 5, Adreno 840) with four models and nine policies, on quality, throughput, heat and energy at once. This paper makes five contributions.

- (i) The realizability gap: nominal per-head budgets do not compact on a sequence-level engine, measured for five policies on four models (§?, §?).
- (ii) In-graph scoring, one keep set and in-place compaction, which keep an attention-driven evictor on the FlashAttention path and make an infeasible configuration run (§3).
- (iii) A thermal watchdog with CPU and GPU ladders anchored at the measured vendor trigger, and the negative result that cache size does not cap peak temperature (§?, §?).
- (iv) An energy-aware scheduler with one lever and two loops, measured through a real battery discharge and under provoked disturbances (§?).
- (v) A multi-axis on-device evaluation with every baseline in its own published configuration (§4).

2 BACKGROUND

H2O [1] keeps heavy hitters by accumulated attention plus recent tokens and reports a 5 to 10 times smaller cache at a 20% budget.

TOVA [3] drops the least-attended position per layer every step and reports full quality at one eighth of the cache. SnapKV [4] freezes a per-head top- K at the end of prefill and reports an 8.2 times smaller cache at 16K inputs. StreamingLLM [2] keeps 4 sinks and a sliding window with no scores and states that it adds no long-term memory. Ada-KV [6] allocates one layer budget across heads and needs FlashAttention-2's variable-length kernel [11] plus a custom CUDA kernel, both absent on the mobile stack. FlashAttention [10] fuses softmax into the kernel and never writes the attention tensor, so a policy that reads scores is locked to the slow path. llama.cpp [14], like every on-device engine we know of, stores the cache as one array of position-indexed cells shared across heads and layers. It removes cells by position and compacts through a state API that restores into a second context. Neither fact is visible on a server with per-head paged storage.

On the control side, zTT [18] and FUSE [19] govern frequency from temperature or phase, with no cache or battery signal. The vendor engine on our SoC deep-throttles the prime cores to 883 MHz the moment the battery reaches 50 °C and caps the GPU at 726 MHz when DDR passes 64 °C. No shipped phone changes an LLM's clock, context or answer length by battery state. EnerInfer [25], the closest research system, excludes battery level by design. Kim, Imes and Hoffmann [24] prove that an energy-optimal schedule under a deadline uses at most two configurations. JouleGuard [23] couples two loops through model error. μ KV's scheduler applies both.

3 μ KV DESIGN

Scoring inside the FlashAttention graph. The fused kernel never materializes attention. μ KV adds a `kq_evict` side node to the prefill graph instead of leaving the fast path. The node computes $\text{softmax}(KQ^T)$ for the last 16 queries of the last prefill chunk and reduces it to one score per layer, head and position, $\bar{A}_{l,h,p}$. It is a graph output, so the mid-graph interception that corrupts this Vulkan backend cannot touch it. It costs 1.2% of prefill on the CPU (paired, $n=29$) and 138 against 131 s on the GPU. Selection is bit-identical to a FlashAttention-off capture.

One keep set for all heads and layers. Selection is sequence-level. The position score is the mean of $\bar{A}_{l,h,p}$ over layers and heads, max-pooled over 7 positions so a run of related tokens survives together (SnapKV's clustering step). A per-prompt split α_a , the fraction of attention mass outside the recent window (floored at 0.70, capped at 0.95), sets how many of the K cells are distant anchors; the rest is the recent window. The top n_{anchor} positions by pooled score survive, plus four sink tokens [2]. The result is one set of K positions shared by every head and layer, which is what lets the cache compact. On the 9737-token prompt $\alpha_a = 0.71$, so $K=1024$ is 4 sinks, 721 anchors and 299 recent cells. Earlier versions also sized a per-head budget from each head's peak attention through a closed-form ramp. Its candidate pool holds 96.5 to 99.8% of the prompt at $K=1024$ on every cell. Bypassing it leaves the keep set identical (721 of 721 positions) and the output identical, so it is removed. K itself is sized from the request's demand, the larger of a retrieval floor and a generation floor. In a controlled test the budget that still retrieves a fact scales with the generated span, from 256 cells at 64 tokens to 4096 at 2048, not with the prompt.

In-place compaction and decode. Compaction through the state API doubles peak cache memory at the instant of restore. Phi-3-mini at 16K needs 2×6144 MiB plus 2.4 GB of weights, and Android killed the process and six co-resident apps, twice. `endurkv_compact_seq()` instead slides survivors into a dense prefix in bounded chunks inside the tensors prefill already allocated. Peak memory does not rise, the configuration runs at 874 cells, and the two modes produce identical keep sets (perplexities agree to 0.15% across four model families). Decode runs over the K -cell cache with the fused kernel. The keep set stays frozen while the recent window slides. A re-eviction fires when the live cache exceeds $1.25\times$ its budget, which bounds the decode mean near 2K cells on a 4K generation.

Reduce-only clock watchdog. A daemon samples skin, battery, DDR and CPU temperature at 2 Hz and lowers a clock cap one step at a time, never raising it. The ladders are anchored at the measured vendor triggers. Battery 47.0 to 49.5 °C and skin 50.0 to 52.5 °C map to CPU caps of 1497 down to 1017 MHz, above the 883 MHz cliff the vendor applies at 50 °C. DDR 60, 62 and 63.5 °C map to GPU caps of 1050, 967 and 902 MHz, below the vendor's 726 MHz trip at 64 °C. The kernel applies the lower of the user cap and the vendor cap, so the ladders act earlier than the vendor and never later. On a cold phone they stay dormant.

Energy-aware scheduling: one lever, two loops. Once per request, in 0.3 s, the scheduler reads the battery level, the charging state, the prompt length, and whether the caller fixed an answer length. A tier follows (mains or above 50%, 21 to 50%, 20% or below) and sets a lever $L \in \{1, 0.5, 0\}$ plus a learned bias. L fixes every rate the scheduler trades at. The exchange rate a step must beat is 1.5, 0.8 or 0.5 J saved per second given up. A quality floor and a time slack follow it, and so does the answer cap when the length is open: 4096, 1024 or 512 tokens. A measured cost table predicts energy and time per plan. A plan is a GPU clock cap for prefill, one for decode and the cache budget. The walk down the ladder takes a step only if the predicted saving beats the exchange rate inside the time budget. After the request, if time ran more than 5% over the budget the plan was chosen under, the performance loop moves the bias 0.1 toward performance. If energy ran more than 5% over its prediction, the energy loop moves it 0.1 toward energy. When both are met the bias decays toward the tier default. The table learns by an exponential moving average clipped to 10% per request. There is no online policy learning on the user's phone. A contextual bandit trained on 24 real requests chose the rule's plan in 13 of 15 free choices, at a cost of 15.7 kJ and 149 minutes of phone time the rule does not pay.

Backend and per-model costs. The table holds CPU plans beside the GPU ones, and they lose on both axes, so the walk never picks the CPU while the GPU is usable. The CPU is the fallback: no Vulkan library, a caller that forces it, or a model that runs on the GPU and produces nothing usable. PrismML's 1-bit Bonsai-8B is the last case, since its Adreno kernels return non-finite logits while decoding at full speed. The scheduler grades each run's output, marks such a model, and re-runs on the CPU. There the clock is not a lever and the cache does not move, because $K=512$ falls below every tier's quality floor, so the answer cap carries the tiering

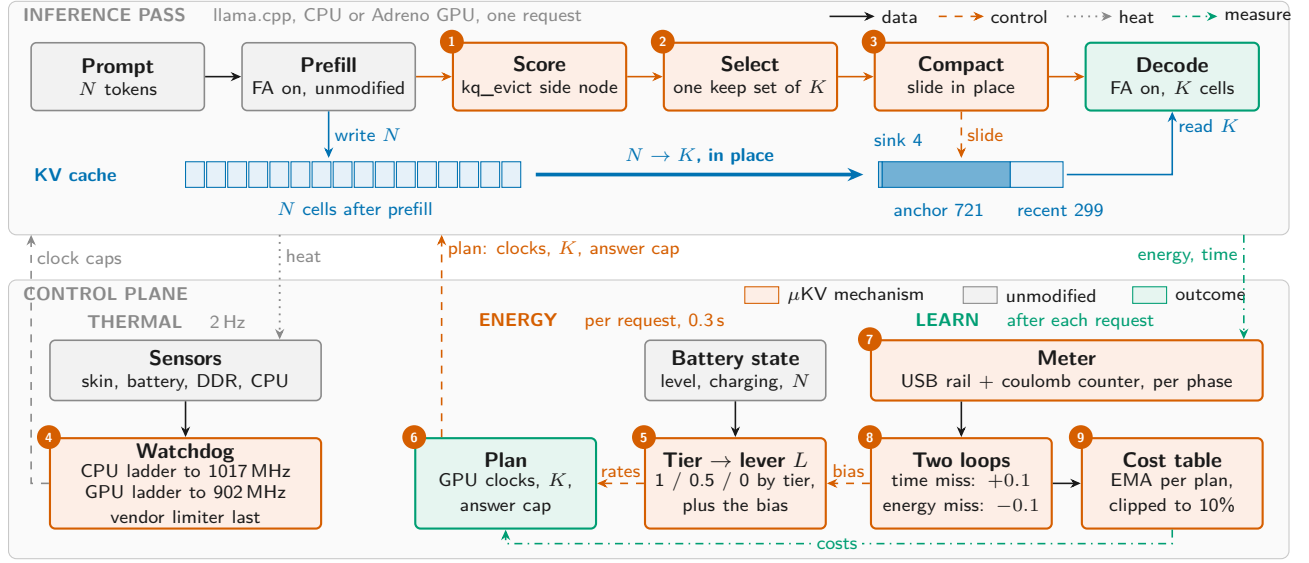


Figure 1: μKV on the phone. Top: the inference pass, with scoring, selection and compaction (1 to 3) as the only changes. Bottom: the thermal, energy and learning columns of the control plane (4 to 9).

alone. Costs are kept per model. Bonsai-8B on the CPU costs 5.1 times what the Llama-3.2-1B row predicts. One shared table would mispredict every fallback, and learning from it would corrupt the row that was right.

4 EVALUATION

Setup. OnePlus 15: Snapdragon 8 Elite Gen 5, 16 GB, Adreno 840. Models: Llama-3.2-1B, Phi-3-mini-128k and gemma-2-2b-it in Q4_K_M, and PrismML’s 1-bit Bonsai-8B [22]. Every timed cell starts behind a cooling gate ($\text{DDR} \leq 35^\circ\text{C}$, $\text{battery} \leq 33^\circ\text{C}$) with charging off, on one build, under the vendor’s own DVFS and thermal engine. Baselines run their own papers’ selection rules. The tables use a matched budget of 1024. SnapKV and Ada-KV keep 1024 cells per head (window 64, kernel 5; Ada-KV safeguard 0.2). H2O keeps 1024 per head, split evenly between heavy hitters and recent tokens. TOVA keeps 1024 per layer with 4 sinks. StreamingLLM keeps 4 sinks and 1020 recent cells. A second campaign runs every baseline at its own published budget (below). Energy integrates the USB rail and the coulomb counter, split at the prefill and decode boundary. μKV runs with $K=1024$ and the watchdog throughout.

Sustained decode on the CPU. Table 1 runs a 9.7K-token prompt followed by 4096 generated tokens. Every policy builds the full prompt cache during prefill (Figure 2). μKV then compacts to 721 cells, and its re-eviction keeps the cache bounded near 2K. SnapKV can only drop to 5.9K, and vanilla grows without bound. On Llama-3.2-1B the bounded cache buys 24.2 against 5.0 tok/s, 2.6 times less wall time and 62% less energy. Its DDR peak is the coolest, 5°C below vanilla, at the same vendor-clamped 1.6 GHz. SnapKV, Ada-KV, H2O and TOVA keep 3.5K to 6.1K cells live at a nominal 1024. That is a 1.6 to 2.8 times reduction, against the 5 to 10 times their papers report. At their own published budgets the gap widens. On 110 LongBench prompts on the phone CPU, SnapKV at 2048 per

Table 1: Sustained decode on the CPU, cold start. Upper two blocks: 9737-token prompt, 4096 generated, $K=1024$. Lower two: the same text (7542 and 6382 tokens), 2048 generated, baselines at their published budgets.

Policy	Budget	tok/s	Wall (s)	Live cells	DDR ($^\circ\text{C}$)	mWh
<i>Llama-3.2-1B</i>						
vanilla (full cache)	1024	5.0	1055	9737	59.8	1033
μKV	1024	24.2	406	721 (1980)	54.4	391
SnapKV	1024	6.8	902	5929	56.3	818
Ada-KV	1024	6.7	875	3484	57.1	1002
StreamingLLM	1024	6.6	877	777	56.7	1052
H2O	1024	5.8	1006	6110	54.8	1094
TOVA	1024	6.0	947	4091	54.4	1069
<i>Bonsai-8B, 1-bit weights</i>						
vanilla (full cache)	1024	1.0	5137	10074	72.2	5758
μKV	1024	4.5	2012	789 (1993)	62.5	2312
SnapKV	1024	1.1	4713	10015	67.9	5476
Ada-KV	1024	1.6	3718	6514	65.2	3971
StreamingLLM	1024	1.6	3777	858	63.7	4012
H2O	1024	1.5	3900	9587	63.3	4136
TOVA	1024	1.6	3841	7116	67.2	4581
<i>Phi-3-mini, 7542-token prompt</i>						
vanilla (full cache)	1024	1.2	2063	7542	60.6	2046
μKV	1024	5.5	841	929	56.3	926
SnapKV	1024/head	1.7	1980	7130	66.0	2530
Ada-KV	2048	1.7	1908	7086	66.8	2479
StreamingLLM	4+2000	2.0	1493	2000	57.1	1311
H2O	20%	1.7	2305	7542	62.5	2660
TOVA	2048	1.7	1856	6425	62.9	2380
<i>gemma-2-2b-it, 6382-token prompt</i>						
vanilla (full cache)	1024	3.9	735	6382	58.7	974
μKV	1024	9.3	427	804	58.7	589
SnapKV	1024/head	4.9	913	6297	60.6	1005
Ada-KV	2048	5.0	909	6222	60.6	989
StreamingLLM	4+2000	4.9	654	2000	57.9	751
H2O	20%	3.9	1017	6382	56.0	1071
TOVA	2048	3.9	1022	6060	56.0	1063

head keeps 94% of the cache and H2O at 20% of the prompt keeps 92%. TOVA and Ada-KV at 2048 keep 77% and 71%. μKV keeps 14%. They pay the FA-off capture as well. They run at 5.8 to 6.8 tok/s

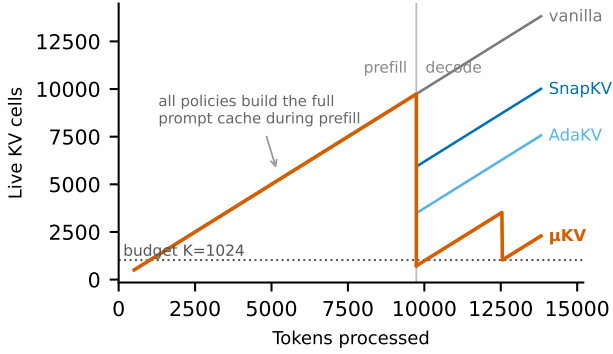


Figure 2: Live KV cells through prefill and decode on Llama-3.2-1B. Only μ KV compacts to its budget and stays bounded.

Table 2: Adreno 840, 12K-token prompt plus 4096 generated tokens, f16 KV for every policy, cold start, median of n runs. Phi-3 rows come from the current build with the driver gate.

Policy	n	Prefill (s)	tok/s	dec $\times$	Cells
<i>Llama-3.2-1B, ctx 16384</i>					
vanilla (full cache)	3	131	24.18	1.00	9741
μ KV	3	138	29.77	1.23	723
μ KV, no compaction	3	132	25.61	1.06	723
SnapKV	1	229	4.76	0.20	6592
Ada-KV	1	276	2.86	0.12	3519
TOVA	1	176	4.34	0.18	4125
H2O	1	279	3.58	0.15	6110
StreamingLLM	1	176	5.55	0.23	777
<i>Phi-3-mini, ctx 16384</i>					
vanilla (full cache)	1	471	5.05	1.00	11157
μ KV, in place	1	586	13.36	2.64	878
StreamingLLM, own budget	1	445	10.60	2.10	2000
Ada-KV, gate promoted	1	572	9.13	1.81	4326
SnapKV, gate promoted	1	532	5.80	1.15	10862
H2O, TOVA	refused by the gate; their CPU rows stand				

and 818 to 1094 mWh, at or above vanilla’s energy. StreamingLLM is the control: it compacts to 777 cells, fewer than μ KV, yet runs at 6.6 tok/s because it stays on the FA-off path in this engine. Retention does not predict speed; the kernel path does. Quality is unchanged. Teacher-forced perplexity on a WikiText slice verified disjoint from the prompt gives 8.198 against 8.023 (vanilla against μ KV) on Llama-3.2-1B. It gives 3.983 against 3.938 on Phi-3-mini and 6.717 against 6.736 on Bonsai-8B. μ KV had evicted 92% of the prompt cache. Weight compression shrinks the static term of per-token traffic; eviction shrinks the growing one. On 1-bit Bonsai-8B (lower block) μ KV gives 4.5 times the throughput, 60% less energy and a DDR peak 9.7 °C cooler, with no change to its parameters. The lower two blocks run the same text on Phi-3-mini and gemma-2-2b with every baseline at its published budget. There μ KV decodes 4.5 and 2.4 times faster than the full cache with 55% and 40% less energy, while the baselines keep 85 to 100% of the cache.

Mobile GPU. Table 2 runs the same workload on the Adreno 840. The CPU path’s state-swap and state-API defrag are not visible here: the swap serializes hundreds of megabytes through host memory, and the Vulkan backend stores FA-off and FA-on caches

Table 3: Quality, both benchmarks measured on the phone. Needle: hits out of 14 depths per model, and the mean live cache after prefill. LongBench: Llama-3.2-1B on the CPU, 103 prompts, each policy at its own published budget.

Policy	Needle: hits / 14, and cells attended					LongBench	
	Phi-3	Llama	Gemma	Bonsai	cells	F1	cache
vanilla	14	13	11	14	4858	36.9	100%
μ KV	14	13	11	14	774	37.0	14%
SnapKV	14	12	11	14	4237	38.0	94%
Ada-KV	14	12	11	14	3361	37.8	71%
TOVA	14	12	11	14	3670	37.4	77%
H2O	14	8	10	13	4212	36.2	92%
StreamingLLM	3	3	2	3	890	35.0	33%

in different layouts. In-graph scoring removes both needs. Prefill is vanilla-class by construction. μ KV decodes at 1.23 times the full cache at 7.4% of the cells; without compaction the cache stays physically sparse and the gain falls to 1.06. On Llama-3.2-1B every per-head evictor runs at 0.12 to 0.23 times the full cache with 3.5K to 6.6K cells live. On Phi-3-mini the round trip is killed at 16K, while in-place compaction runs there and reaches 2.64 times the full cache on 7.9% of the cells. Phi-3 also needs a driver gate: the FlashAttention-off capture is numerically broken at head dimension 96, and uncorrected runs decode at full speed while emitting 18 to 44% corrupted tokens. The gate promotes SnapKV and Ada-KV to the in-graph side node and refuses H2O and TOVA. The two promoted evictors then reach 1.15 and 1.81 times the full cache while holding 97% and 39% of it. Quantized KV is numerically invalid on this driver, so every GPU row is f16.

Retrieval and long-context quality. A fact is buried at one of seven depths in 4K or 8K tokens of filler. The 392 cells cover seven policies, four models, seven depths and two contexts (Table 3, left). μ KV matches the full cache on every model while attending to 774 cells, against 3361 to 4237 for the score-reading evictors, which is 69 to 87% of the full cache. StreamingLLM fails retrieval everywhere; its window evicts exactly the middle of the context. Where every policy misses a depth, the full cache misses it too. This workload generates 64 tokens, so it is prefill-dominated and says nothing useful about speed or energy; those are the 4096-token workload of Table 1. LongBench (Table 3, right) runs on the phone CPU with Llama-3.2-1B, on 103 prompts, each policy at its own published budget. μ KV scores 37.0 against the full cache’s 36.9 while holding 14% of the cache. SnapKV, Ada-KV and TOVA gain 0.6 to 1.1 F1 while holding 71 to 94%, and StreamingLLM loses 1.9 at 33%. What separates the policies is the cache each one frees, not the budget it asks for. A four-model, five-task sweep on a desktop GPU gives the same ordering at 12 to 13% of the cache. There μ KV lands within 0.1 F1 of the full cache on Llama-3.2-1B and Phi-3-mini, and 2.8 and 3.7 below it on gemma-2-2b and Bonsai-8B.

Sustained heat and the watchdog. Figure 4 shows the workload that reaches the cliff. The full cache on Bonsai-8B runs 85.6 minutes and drives the battery to 50.2 °C. At 50 °C the vendor deep-throttles both clusters, the prime cores to 883 MHz, in a sawtooth of 17 dips that spends 923 s at 883 MHz in total. μ KV finishes in 33.2 minutes at a 44.2 °C battery peak, before the trigger. The watchdog belongs

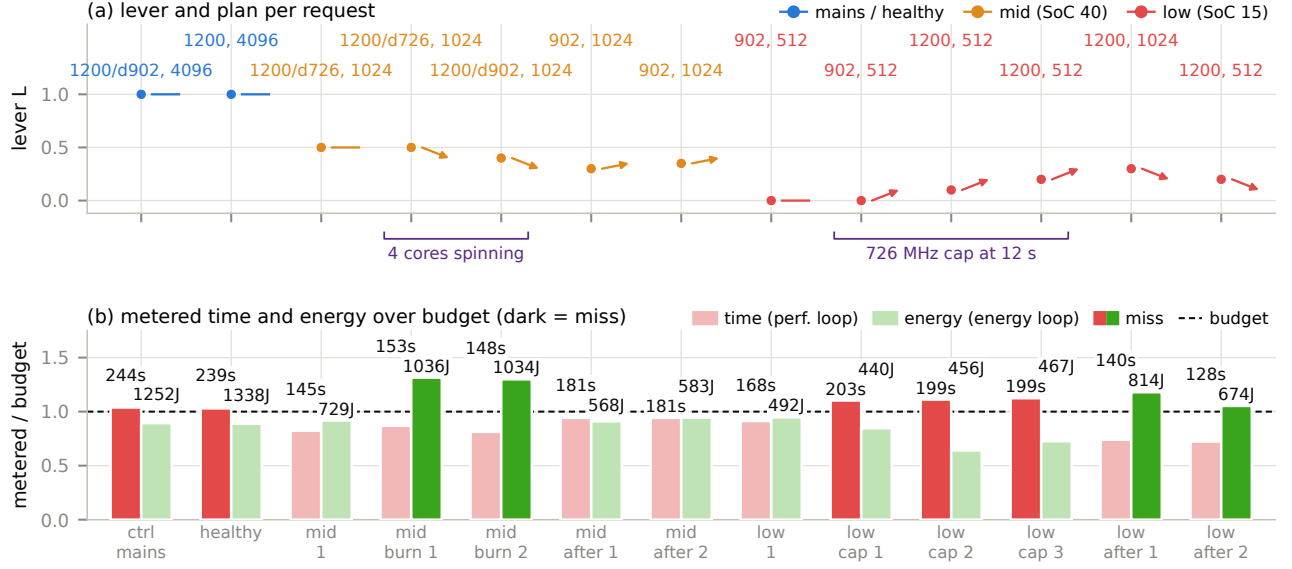


Figure 3: The two loops provoked on the phone: the lever and plan of thirteen cooled requests (top), and their metered time and energy against budget (bottom). Dark bars are misses.

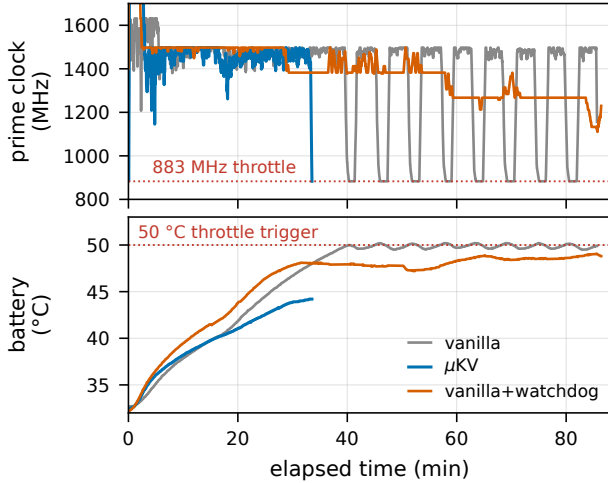


Figure 4: Bonsai-8B on the CPU from a cold start: prime clock and battery for the full cache, μKV, and the full cache with the watchdog (ablation).

to μKV and is never given to a baseline; the full-cache trace with it is an ablation, since μKV finishes before the cliff. Given it, the full cache’s battery rise flattens from 0.48 to 0.05 °C/min and settles at 49.1 °C. The prime cores never reach 883 MHz. This decode is bandwidth-bound, so the throttle costs no speed (1.0 tok/s either way): the watchdog buys sustainability, and μKV buys speed. On the GPU, three cooled cells per arm of the 4096-token request give 1289 against 1390 J and a DDR peak of 75 against 90 °C for 1% more

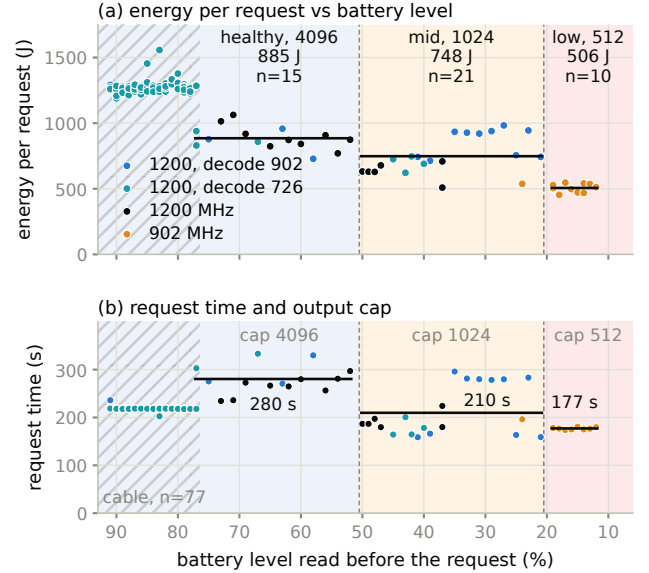


Figure 5: The scheduler through a real discharge: energy and time per request against the battery level it read, with tier bands, the plan chosen and the answer cap. Hatched: cable in.

time. We also tied the cache budget itself to DDR temperature, a four-tier ladder over $K \in \{256, \dots, 1024\}$. The result is negative. Halving K under co-load left peak DDR unchanged, and on a cool

Table 4: The plan at each battery tier and its cost on the cable (cooled cells) and on the battery (the real discharge). Llama-3.2-1B, 9737-token prompt, GPU.

Battery	GPU plan	Cap	Cable J	Batt. J	Batt. s
mains or above 50%	1200, decode 902	4096	1336	885	280
21 to 50%	1200, decode 726	1024	577	748	210
20% or below	902	512	500	506	177
GPU unavailable	CPU, $K=1024$	tier	822	.	.

phone a smaller cache runs hotter at peak, because the saved bandwidth converts to speed and power. Cache size controls throughput, time and energy; the clock caps heat.

Energy-aware scheduling. Every number here is from the GPU with Llama-3.2-1B and a 9737-token prompt, cooled per cell. The GPU clock cap trades prefill energy for time. From 1200 to 902 MHz a 4096-token request falls from 1428 to 1154 J and rises from 221 to 263 s. The next step to 726 MHz buys 3% more energy for 21% more time, so it is never taken. Decode barely feels the cap, so the split plan, prefill at 1200 and decode at 902, saves 6% of the request for 1.5% of time. The CPU clock is not an energy lever: energy per token is flat within 7% from 883 to 1632 MHz. Below $K=1024$ the cache saves 3 to 4% while the worst task loses 37 to 42% of its score. Output length is linear at 0.17 to 0.21 J per token, which makes the answer cap the largest lever when the length is open. Thirteen cooled requests with real disturbances then exercised the loops (Figure 3). Four idle cores spinning through a mid-tier request pushed its energy 30% over budget; the energy loop took the lever down twice, and two clean requests decayed it back. An external 726 MHz cap 12 s into three low-tier requests, which is what the vendor limiter does, put them 10 to 12% over their time budget. The performance loop took the lever up to its clamp. The energy loop returned it once the cap was gone.

A loop resident on the phone then ran 123 requests through the scheduler from 91 to 11%, with charging disabled and no forced state (Figure 5, Table 4). The scheduler switched on the first request past each threshold. On the battery the tiers cost 885, 748 and 506 J per request in 280, 210 and 177 s: 15% and 43% less energy than the healthy tier, with time falling 25% and 37%. On the battery the phone is a different machine from the phone on the cable. The same healthy request costs 885 J instead of 1274, because the OEM runs the GPU cooler when unplugged and caps prefill at 726 MHz within a minute. There the answer cap and the prefill cap carry the saving. A table seeded from cable cells over-predicted battery requests by 36%; the clipped learning brought it within 2% in three requests. The mean prediction error over the run was 8.9% on the battery against 3.7% on the cable. PowerBench [26] and the FUSE study of governors under LLM inference [27] agree with our per-token costs and with the finding that a profiled static setting matches learned governors.

Scope. All measurements are on one OnePlus 15. The ladder thresholds are device-specific; the calibration protocol generalizes. Energy is total-device energy from the rail and the coulomb counter, so absolute joules hold within about 10% while orderings hold. Eviction still costs verbatim recall of the spans it drops, even though prediction and retrieval are preserved. The 1-bit kernels

return non-finite logits on Adreno, so Bonsai’s phone results are CPU results.

5 RELATED WORK

Per-head budget allocation is Ada-KV’s idea [6]; we credit it and do not reprove its bound. CriticalKV [7], DefensiveKV [8] and AhaKV [9] refine the score. R-KV [20] and DynamicKV [21] argue for task-aware budgets, as our demand-based sizing does. KIVI [5] and KVzip [15] are orthogonal. None reports a temperature, a watt or a resident set on a phone. Among mobile systems, PowerInfer-2 [12] and LLM in a Flash [13] optimize weight access. KVSwap swaps the cache to disk on a Jetson, and DynaKV offloads to UFS on the CPU. PerCache [16] shortcuts whole passes at the application level and composes with μ KV. MLC-LLM [17], ExecuTorch and the vendor SDKs use the unmodified cache. Verified from their evaluation sections: the policies that evict run on servers or Jetson boards. The systems that run on phones keep every token. The studies that use a phone GPU apply no cache management. μ KV occupies the empty cell. On control, zTT [18] and FUSE [19] act on frequency alone. We know of no prior system that reads attention-internal state and battery and thermal sensors in one decode loop on a phone.

6 CONCLUSION

μ KV makes attention-driven KV eviction work on a phone by respecting the engine. Scores come from inside the FlashAttention graph. One keep set is what the shared cell array can free. Compaction never needs a second context. A reduce-only clock watchdog keeps the vendor throttle unreachable, and a per-request scheduler with one lever and two loops spends the battery according to its level. On a OnePlus 15 the result is 4.8 times the full cache’s decode throughput on the CPU and 1.23 times on the GPU, at 7% of the cells and 62% less energy. Retrieval matches the full cache and LongBench nearly does. A real discharge’s lower tiers cost 15% and 43% less per request. The cache is the phone’s efficiency actuator and the clock its temperature actuator. Every per-head budget that looks good on a server is a full cache in disguise on the device.

REFERENCES

- [1] Z. Zhang et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs.” *NeurIPS*, 2023.
- [2] G. Xiao et al. “Efficient Streaming Language Models with Attention Sinks.” *ICLR*, 2024.
- [3] M. Oren et al. “Transformers are Multi-State RNNs.” *arXiv:2401.06104*, 2024.
- [4] Y. Li et al. “SnapKV: LLM Knows What You’re Looking For Before Generation.” *NeurIPS*, 2024.
- [5] Z. Liu et al. “KIVI: Tuning-Free Asymmetric 2bit Quantization for KV Cache.” *arXiv:2402.02750*, 2024.
- [6] Y. Feng et al. “Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation.” *NeurIPS*, 2025.
- [7] Y. Feng et al. “CriticalKV: A Perturbation Perspective on KV Cache Eviction.” *ICML*, 2026.
- [8] Y. Feng et al. “DefensiveKV: Worst-Case Risk Control for KV Cache Eviction.” *ICLR*, 2026.
- [9] Y. Gu et al. “AhaKV: Adaptive Holistic Attention-Driven KV Cache Eviction.” *arXiv:2506.03762*, 2025.

- [10] T. Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention." NeurIPS, 2022.
- [11] T. Dao. "FlashAttention-2: Faster Attention with Better Parallelism." arXiv:2307.08691, 2023.
- [12] Y. Song et al. "PowerInfer-2: Fast LLM Inference on a Smartphone." arXiv:2406.06282, 2024.
- [13] K. Alizadeh et al. "LLM in a Flash: Efficient Inference with Limited Memory." arXiv:2312.11514, 2023.
- [14] G. Gerganov. "llama.cpp." <https://github.com/ggerganov/llama.cpp>.
- [15] J.-H. Kim et al. "KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction." NeurIPS, 2025.
- [16] K. Liu et al. "PerCache: Predictive Hierarchical Cache for RAG Applications on Mobile Devices." arXiv:2601.11553, 2025.
- [17] MLC Team. "MLC-LLM: Universal LLM Deployment." <https://github.com/mlc-ai/mlc-llm>.
- [18] S. Kim et al. "zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices." MobiSys, 2021.
- [19] S. Liu et al. "FUSE: Fine-Grained Power Management for Mobile Devices." SenSys, 2022.
- [20] X. Cai et al. "R-KV: Redundancy-aware KV Cache Compression for Reasoning Models." arXiv:2505.24133, 2025.
- [21] X. Zhou et al. "DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs." arXiv:2412.14838, 2024.
- [22] PrismML. "Bonsai: Commercially Viable 1-bit LLMs." 2026. <https://prismml.com/news/bonsai-8b>.
- [23] H. Hoffmann. "JouleGuard: Energy Guarantees for Approximate Applications." SOSP, 2015.
- [24] D. H. K. Kim, C. Imes, and H. Hoffmann. "Racing and Pacing to Idle: Theoretical and Empirical Analysis of Energy Optimization Heuristics." CPSNA, 2015.
- [25] B. Zou et al. "EnerInfer: Energy-Aware On-Device LLM Inference." arXiv:2606.23001, 2026.
- [26] G. Cai et al. "Is Your NPU Ready for LLMs? A Power Benchmark of On-Device Inference." arXiv:2607.05475, 2026.
- [27] Z. Zhang et al. "Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency." arXiv:2507.02135, 2025.